

University of Central Missouri
Department of Computer Science & Cybersecurity

CS5760 Natural Language Processing
Fall 2025

Homework 4.

Student name:Duggineni Sesha Rao

Submission Requirements:

- Once finished your assignment push your source code to your repo (GitHub) and explain the work through the ReadMe file properly. Make sure you add your student info in the ReadMe file.
- Submit your GitHub link on the Bright Space.
- Comment your code appropriately ***IMPORTANT***.
- Any submission after provided deadline is considered as a late submission.

Part I. Short Answer

1) RNN Families & Use-Cases (Many-to-X)

a) Map each task to the most suitable RNN I/O pattern and explain why (1–2 lines each):

- next-word prediction

Suitable RNN I/O Pattern:

Many-to-one: Model reads a sequence of previous words (many inputs) and predicts the next single word (one output).

- sentiment of a sentence

Suitable RNN I/O Pattern:

Many-to-one: The model processes an entire sentence and outputs a single sentiment label.

- NER

Suitable RNN I/O Pattern:

Many-to-many (aligned): Each input word has a corresponding output tag (e.g., PER, LOC, O), so outputs align one-to-one with inputs.

- machine translation

Suitable RNN I/O Pattern:

Many-to-many (unaligned): Input and output sequences differ in length (e.g., English → French), so outputs are not aligned word-for-word.

(Choose from: one-to-many, many-to-one, many-to-many aligned, many-to-many unaligned.)

b) In one sentence, explain how “unrolling” over time enables BPTT and weight sharing.

Unrolling over time expands the RNN into a sequence of repeated layers (one per time step), allowing **Backpropagation Through Time (BPTT)** to compute gradients across all steps while **sharing the same weights** at each time step.

c) Give one advantage and one limitation of weight sharing across time in RNNs.

☐ **Advantage:** Enables the model to generalize across sequence positions with far fewer parameters.

☐ **Limitation:** Makes it harder to learn long-term dependencies because the same weights are reused, often leading to **vanishing or exploding gradients**.

2) Vanishing Gradients & Remedies

a) Describe the vanishing gradient problem in RNNs and how it affects long-range dependencies.

The **vanishing gradient problem** occurs when gradients shrink exponentially as they are backpropagated through many time steps, causing early layers to learn very slowly — this prevents RNNs from capturing **long-range dependencies** in sequences.

b) List two *architectural* solutions and briefly how each helps gradient flow

□ **LSTM (Long Short-Term Memory)**: Uses gates and a cell state to preserve and regulate information flow, preventing gradients from vanishing.

□ **GRU (Gated Recurrent Unit)**: Simplified gated design that maintains gradient flow through reset and update gates, reducing vanishing effects.

c) Give one *training* technique (not an architecture change) that can mitigate the issue and why.

Gradient clipping — limits the magnitude of gradients during training to prevent them from becoming too small or too large, stabilizing learning and helping preserve gradient flow.

3) LSTM Gates & Cell State

a) Explain the roles of the forget, input, and output gates. For each, name its activation and purpose.

Forget Gate: Activation Function: Sigmoid (σ)

Role: Decides which parts of the previous cell state to discard.

Input Gate: : Activation Function: Sigmoid + Tanh

Role: Controls which new information to add to the cell state.

Output Gate: Activation Function: Sigmoid + Tanh

Role: Determines what part of the cell state to output as hidden state.

b) Why is the LSTM cell state often described as providing a “linear path” for gradients?

The **LSTM cell state** provides a **linear (additive) path** for gradients to flow through time, reducing gradient decay and allowing long-term information to be preserved — this is why it mitigates the vanishing gradient problem.

c) In one or two sentences, contrast “what to remember” vs. “what to expose” in LSTMs.

□ **“What to remember”** is controlled by the **forget and input gates**, deciding which information stays in the cell state.

□ **“What to expose”** is controlled by the **output gate**, deciding which parts of that stored information influence the current output.

4) Self-Attention

a) Define Query (Q), Key (K), and Value (V) in the context of self-attention.

□ **Query (Q):** A vector representing the item for which we are computing attention; it asks *“how much should I focus on each element?”*

□ **Key (K):** A vector representing each element in the sequence that the query will compare against.

□ **Value (V):** A vector containing the information to be aggregated based on the attention scores.

b) Write the formula for dot-product attention.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) \cdot V$$

c) Why do we divide by $\sqrt{d_k}$?

Dividing by $\sqrt{d_k}$ **prevents the dot products from growing too large** when the dimensionality is high, which would otherwise push the softmax into regions with very small gradients, slowing learning.

5) Multi-Head Attention & Residual Connections

a) Why do Transformers use multi-head attention instead of single-head attention?

Transformers use **multi-head attention** to allow the model to **attend to information from different representation subspaces and positions simultaneously**, capturing richer relationships than a single attention head could.

b) What is the purpose of Add & Norm (Residual + LayerNorm)? Explain two benefits.

□ **Purpose:** Combines the input of a sub-layer with its output (residual connection) and normalizes it (layer normalization) for stable training.

□ **Benefits:**

1. **Improves gradient flow** during backpropagation, helping train deep networks.
2. **Stabilizes and speeds up training** by preventing covariate shift in intermediate layer activations.

c) Describe one example of linguistic relation that different heads might capture (e.g., coreference, syntax).

Different heads can capture relations such as **coreference** (linking pronouns to their antecedents), **syntactic dependencies** (subject–verb agreement), or **positional relationships** between words.

6) Encoder–Decoder with Masked Attention

a) Why does the decoder use masked self-attention? What problem does it prevent?

The decoder uses **masked self-attention** to prevent each position from attending to **future tokens** in the output sequence, ensuring the model only has access to **previously generated tokens** and avoids “cheating” during training.

b) What is the difference between encoder self-attention and encoder–decoder cross-attention?

□ **Encoder self-attention:** Each position in the input sequence attends to **all positions in the input** to build contextualized representations.

□ **Encoder–decoder cross-attention:** Each position in the decoder attends to **all encoder outputs** to incorporate information from the source sequence while generating the output.

c) During inference (no teacher forcing), how does the model generate tokens step by step?

Without teacher forcing, the model **generates tokens autoregressively**: it predicts one token, appends it to the output sequence, and uses the updated sequence as input to predict the next token, repeating until an end-of-sequence token is generated.

Part II: Programming

Q1. Character-Level RNN Language Model (“hello” toy & beyond)

Goal: Train a tiny character-level RNN to predict the next character given previous characters.

Data (toy to start):

- Start with a small toy corpus you create (e.g., several “hello...”, “help...”, short words/sentences).
- Then expand to a short plain-text file of ~50–200 KB (any public-domain text of your choice).

Model:

- Embedding → RNN (Vanilla RNN or GRU or LSTM) → Linear → Softmax over characters.
- Hidden size 64–256; sequence length 50–100; batch size 64; train 5–20 epochs.

Train:

- Teacher forcing (use the true previous char as input during training).
- Cross-entropy loss; Adam optimizer.

Report:

1. Training/validation loss curves.
2. Sample 3 temperature-controlled generations (e.g., $\tau = 0.7, 1.0, 1.2$) for 200–400 chars each.
3. A 3–5 sentence reflection: what changes when you vary sequence length, hidden size, and temperature?
(Connect to slides: embedding, sampling loop, teacher forcing, tradeoffs)

Q2. Mini Transformer Encoder for Sentences

Task: Build a mini Transformer Encoder (NOT full decoder) to process a batch of sentences.

Steps:

1. Use a small dataset (e.g., 10 short sentences of your choice).
2. Tokenize and embed the text.
3. Add sinusoidal positional encoding.
4. Implement:
 - Self-attention layer
 - Multi-head attention (2 or 4 heads)
 - Feed-forward layer
 - Add & Norm
5. Show:
 - Input tokens
 - Final contextual embeddings
 - Attention heatmap between words (visual or printed)

Q3. Implement Scaled Dot-Product Attention

Goal: Implement the attention function from your slides:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Requirements:

- Write a function in PyTorch or TensorFlow to compute attention.
- Test it using random Q, K, V inputs.
- Print:
 - Attention weight matrix
 - Output vectors
 - Softmax stability check (before and after scaling)

